

INTERVIEW PREPARATION GUIDE

PitWall

A Multi-Agent RAG System for Formula 1 Race Strategy

A complete, from-scratch walkthrough of every layer of the PitWall codebase — data pipeline, RAG architecture, multi-agent orchestration, backend API, frontend, database design, deployment, and the engineering trade-offs behind each decision — written so every likely interview question about this project has a grounded, specific answer.

Python · FastAPI

ChromaDB

sentence-transformers

Groq LLM (Llama 3)

SQLite

React 19 · Vite · TypeScript

Tailwind CSS v4

Render · Vercel

Contents

01	Project Overview — what it is, the problem, the elevator pitch	1
02	Tech Stack & Why Each Piece Was Chosen	2
03	System Architecture — the end-to-end request path	3
04	Data Pipeline — FastF1 ingestion, SQLite, caching	4
05	The RAG Pipeline — chunking, embeddings, vector search	5
06	The Multi-Agent System — orchestrator, 6 specialist agents	6
07	Text-to-SQL: the Data Agent	7
08	Backend API — FastAPI routes & design patterns	8
09	Frontend Architecture — React, routing, state	9
10	Database Design — SQLite schema + ChromaDB collections	10
11	Prompt Engineering & Anti-Hallucination Design	11
12	Deployment & DevOps — Render, Vercel, the OOM war story	12
13	Key Engineering Decisions — a curated "why" list	13
14	Glossary — every term, defined plainly	14
15	Interview Q&A Bank — organized by topic	15
16	"What Would You Improve?" — honest limitations	16

Project Overview

The elevator pitch

PitWall is a full-stack AI application that answers natural-language questions about Formula 1 race strategy — tyre choices, pit-stop timing, weather impact, rival threats, circuit incidents — by combining **retrieval-augmented generation (RAG)** over real F1 telemetry with a **multi-agent system** where specialist "agents" each own a slice of the domain (tyres, weather, radio, rivals, circuit, and raw data/SQL), run in parallel, and get synthesized into one final answer by an orchestrator LLM call.

It ships as two deployable halves: a **FastAPI backend** (the RAG/agent engine, deployed on Render) and a **React frontend** (deployed on Vercel) that presents race weekends, standings, a live pre-race "brief" and post-race "debrief", and a chat interface into the agent system.

The problem it solves

Raw F1 timing data (lap times, tyre compounds, pit stops, weather) is abundant but not *answerable* — you can't ask a CSV "why did Verstappen pit on lap 32?". PitWall turns that structured + unstructured data (lap telemetry, weather logs, team radio transcripts) into something a race engineer could actually query conversationally, while staying strictly grounded in retrieved facts rather than the LLM's general knowledge (which would happily hallucinate a plausible-sounding but wrong answer).

What makes it more than "a RAG chatbot"

- **Domain decomposition** — instead of one generic RAG call, six specialist agents each retrieve from a scoped subset of collections and answer with a domain-specific system prompt, then get reconciled by a synthesis step.
- **Hybrid retrieval** — semantic vector search (ChromaDB) for narrative/context questions, **and** a text-to-SQL agent for exact aggregate questions ("how many pit stops did Ferrari make?") that a small sample of retrieved chunks can never answer correctly.
- **LLM-based dynamic routing** — the orchestrator asks an LLM which agents are relevant per question, rather than hardcoding keyword rules.
- **Deliberate anti-hallucination guardrails** — driver-code grounding via a real name legend, an explicit "I don't have enough data" escape hatch, and rules stopping agents from stating unverifiable aggregates from a small retrieved sample.
- **A real deployment story** — a from-scratch Render/Vercel deployment that surfaced (and fixed) a genuine production issue: an out-of-memory crash caused by PyTorch's default CUDA build on Linux. See Chapter 12.

Interview framing: If asked "walk me through a project," the strongest narrative order is: problem → why RAG alone isn't enough → why multi-agent → how routing/synthesis works → one concrete guardrail you added and why → the deployment issue you diagnosed and fixed. That order shows product thinking, system design, and operational maturity in under two minutes.

Tech Stack & Why Each Piece Was Chosen

Backend

LAYER	CHOICE	WHY
Web framework	<code>FastAPI</code>	Async-capable, automatic OpenAPI docs, Pydantic request/response validation for free.
Server	<code>uvicorn[standard]</code>	ASGI server; standard extras add uvloop/httptools for performance.
Vector database	<code>chromadb</code> (PersistentClient)	Embedded, file-backed vector DB — no separate server process to run/deploy; simple <code>get_or_create_collection</code> API.
Embedding model	<code>sentence-transformers</code> — <code>BAAI/bge-small-en-v1.5</code>	Free, runs locally on CPU, 384-dim, strong quality/speed trade-off for RAG — no per-embedding API cost or network dependency.
LLM inference	<code>groq</code> SDK — Llama 3 family	Groq's LPU inference is extremely fast (key for a multi-agent pipeline that fires several LLM calls per question) and has a generous free tier.
Relational store	<code>sqlite3</code> (stdlib) — <code>data/pitwall.db</code>	Zero-ops embedded SQL database for exact, deterministic facts (results, standings-adjacent stats) — no server, ships as a single file.
Data ingestion	<code>fastf1</code>	The de-facto Python library for official F1 timing/telemetry data, with built-in HTTP response caching.
Data wrangling	<code>pandas</code>	CSV parsing, aggregation, timedelta handling for lap times.
Config	<code>python-dotenv</code>	Loads <code>.env</code> locally; no-ops safely in production where real env vars are injected by the platform.

Frontend

LAYER	CHOICE	WHY
Framework	React 19	Component model fits the dashboard + chat UI; huge ecosystem.
Build tool	Vite 8	Fast dev server + optimized production build; native <code>import.meta.env</code> support for environment-based config.
Routing	react-router-dom v7 (<code>BrowserRouter</code>)	Client-side routing for <code>/races/:year/:round</code> style deep links.
Styling	Tailwind CSS v4 + custom design tokens	Utility-first styling with a bespoke dark, telemetry-style visual language.
Language	TypeScript	Typed API contracts (see <code>lib/api.ts</code>) catch backend/frontend shape drift at compile time.

Infrastructure

CONCERN	CHOICE	WHY
Backend hosting	Render (Web Service, free tier)	Git-based deploys, free tier is enough for a portfolio project, simple blueprint (<code>render.yaml</code>) config.
Frontend hosting	Vercel	First-class Vite support, generous free tier, instant global CDN for static assets.
Version control / data	Git, with the built ChromaDB store committed directly	Avoids needing a persistent disk or a rebuild-on-deploy ingestion step (see Chapter 12).

Interview angle: Be ready to justify SQLite + ChromaDB as *two separate stores* rather than one. The honest answer: they solve different problems — SQLite gives exact, provably-correct aggregate answers via SQL; ChromaDB gives semantic recall over free-text/narrative content (radio transcripts, lap descriptions) where there's no fixed schema to query against. Using only vector search for "how many times did X happen" is unreliable because retrieval returns a *sample*, not an exhaustive result set — that exact reasoning shows up as an explicit rule in every agent's system prompt (Chapter 11).

System Architecture — the End-to-End Request Path

Tracing one request — a user asking *"Should Norris have pitted earlier?"* in the Chat page — through the whole system:



Key architectural properties

- **Fan-out / fan-in concurrency** — selected agents run in parallel via `concurrent.futures.ThreadPoolExecutor`, so wall-clock latency is roughly the slowest single agent, not the sum of all of them (I/O-bound HTTP calls to Groq, so threads — not processes — are the right tool; the GIL doesn't matter here).
- **Two-tier model usage** — routing and each specialist's answer use the fast, cheap `llama-3.1-8b-instant`; final synthesis and text-to-SQL generation use the larger, more careful `llama-3.3-70b-versatile` — see Chapter 13 for why.
- **Graceful degradation** — if one agent's thread raises, the orchestrator logs it and continues with whatever results did come back (`_run_agents` catches exceptions per-future); only a *total* wipeout returns the

fallback string.

- **Stateless backend** — no server-side session; every request is self-contained (question + optional race/year). The only mutable server state is the per-race in-memory cache for `/brief` and `/debrief`.

Data Pipeline — Ingestion (offline, not part of the live API)

Everything under `ingest/` is run **once, locally, ahead of time** — the deployed API never calls FastF1 or does any scraping. This separation matters: it keeps the live service's dependency footprint small (see Chapter 12) and its behaviour deterministic (no external API flakiness at request time).

The pipeline, stage by stage

- 1 `fetch_schedule.py` / `fetch_full_schedule.py` — pull the season calendar (rounds, countries, session dates) from FastF1 into `data/schedule_<year>.json`.
- 2 `fetch_laps.py <year> <country>` — loads the Race session via `fastf1.get_session(...).load()`, writes 4 CSVs: laps, pitstops (derived: laps with a non-null `PitInTime`), weather, and a driver code→full-name→team reference (from `session.results` — the *only* place full names exist in the dataset).
- 3 `fetch_quali.py`, `fetch_radio.py` + `transcribe_radio.py` (Whisper), `fetch_standings.py` — supplementary per-race and per-season data.
- 4 `utils/db_manager.py::init_db()` — concatenates every race's CSVs into one flat SQLite DB with tables `laps`, `pitstops`, `weather`, `drivers`, `qualifying` (each tagged with `Race` + `Year` columns).
- 5 `run_all_seasons.py` — for every race: chunk (Chapter 5) → embed → upsert into ChromaDB. `reingest.py` wipes and rebuilds the four collections from scratch.

FastF1's own HTTP cache

`fastf1.Cache.enable_cache("cache/")` is enabled only inside the `ingest/` scripts — it memoizes FastF1's own network calls to the F1 timing API so re-running ingestion doesn't re-download the same session twice. This cache grew to **~2.9 GB** locally and is deliberately `.gitignore`d and *not* part of the deployment — the live API never touches it.

Why a two-stage pipeline (fetch → chunk/embed) instead of one script? Separation of concerns: fetching is slow/network-bound and idempotent per race; chunking/embedding is CPU-bound and re-run whenever the chunking strategy changes. Keeping them as separate scripts invoked by a bulk runner (`run_all_seasons.py`) means a chunking-format change doesn't require re-hitting the F1 API for 29 race weekends.

The RAG Pipeline — Chunking, Embedding, Retrieval

5.1 • Chunking strategy (`utils/chunker.py`)

Each data source gets a chunking strategy matched to its natural grain:

SOURCE	CHUNK GRANULARITY	EXAMPLE TEXT
Laps	One chunk per driver-lap	Driver: VER Lap: 32 Compound: SOFT TyreLife: 5 Position: 1 LapTime: 1:12.345 PitInTime: N/A PitOutTime: N/A
Pit stops	One chunk per pit stop (derived from laps with a pit-in time)	Driver: VER pitted on Lap 32 PitInTime: 1234.5s Switched to: HARD TyreLife going in: 5 laps
Weather	Rolling windows of 10 samples, averaged	AirTemp: 28.5°C TrackTemp: 44.2°C Humidity: 38% Rainfall: False WindSpeed: 1.2
Radio	One chunk per speaker utterance (parsed from <code>[SPEAKER]</code> -tagged transcript blocks)	[GP] Max, we're going to box you this lap. Soft tyres ready.

Every chunk carries a flat metadata dict — always including `source`, `race`, and `year` — which is what makes race/season-scoped filtering possible at query time without re-embedding anything.

5.2 • Embedding (`utils/embedder.py`)

- Model: `BAAI/bge-small-en-v1.5` via `sentence-transformers` — 384 dimensions, runs on CPU, downloaded once from Hugging Face and cached.
- Embeddings are **L2-normalized** (`normalize_embeddings=True`) at encode time specifically so ChromaDB's distance metric behaves as cosine similarity.
- Batched encoding (default batch size 100) for ingestion throughput.

5.3 • Vector store (`utils/vectorstore.py`)

A single `chromadb.PersistentClient(path="vectorstore/")` backs four named collections — `laps`, `weather`, `radio`, `pitstops` — each with its own `SentenceTransformerEmbeddingFunction`. Loading uses **upsert** with deterministic IDs (e.g. `"monaco_2025_VER_lap32"`) so re-running ingestion for a race overwrites rather than duplicates records.

5.4 • Retrieval (`utils/retriever.py`)

```
def retrieve(query, collections=None, top_k=15, distance_threshold=0.80,
            race=None, year=None) -> list[dict]:
    # 1. embed the query with the same model used at ingestion
    # 2. for each requested collection:
    #     build a ChromaDB `where` filter: {"race": race} AND/OR {"year": year}
    #     collection.query(query_embeddings=[...], n_results=top_k, where=where)
    #     keep only hits with distance <= distance_threshold
    # 3. merge all collections' results, sort by distance ascending
    # 4. cap at 30 chunks total
```

Two important, deliberate design choices here:

- **Race/year scoping via metadata filter, not just similarity.** Lap-data chunks are extremely template-heavy across races ("Driver: X | Lap: N | Compound: SOFT..."), so nearest-neighbour search alone can easily surface the right *shape* of chunk from the *wrong* race. Filtering by `{"race": ..., "year": ...}` before ranking eliminates that failure mode entirely rather than hoping similarity sorts it out.
- **Per-agent thresholds differ from the module default.** `BaseAgent._retrieve` calls with `top_k=8, distance_threshold=0.50` (tighter than the retriever's own default of `top_k=15, distance_threshold=0.80`) — because race/year scoping already narrows the candidate pool to one race's data, a smaller, stricter set reliably still contains the right chunk while keeping the per-agent LLM prompt small (bounds token usage/cost/latency across up to 4 parallel agent calls).

What is "distance" here? ChromaDB returns cosine *distance* (not similarity) between the query embedding and each stored embedding — `0` means identical direction, `2` means opposite. Because embeddings are L2-normalized, cosine distance and Euclidean distance are monotonically related, so either framing is valid to discuss. A threshold of `0.50` is a deliberately conservative cutoff — it would rather return nothing (triggering the "I don't have enough data" fallback) than a loosely-related chunk that leads to a fabricated-sounding answer.

5.5 • A note on `utils/rag.py` and `utils/prompt_builder.py`

These two modules implement a complete, simpler, *single-shot* RAG pipeline (retrieve → build one prompt → one LLM call) with source-citation tags like `[laps]` / `[weather]` / `[radio]`. They predate the multi-agent system and are **not used by the live API or agents** today — they only remain referenced by `utils/test_rag.py` and `ingest/verify_expansion.py` as a smoke-test / sanity-check path. Each specialist agent now builds its own prompt inline instead.

Good interview answer if asked "did the architecture change over time?": Yes — it started as one linear RAG call and evolved into a multi-agent design once a single system prompt covering tyres + weather + radio + rivals + circuit context started producing shallower, less reliable answers than dedicated specialists with narrower context and rules per domain. The old code was left in place (not deleted) because it's still a useful minimal reference/smoke test for whether retrieval + generation work in isolation, decoupled from orchestration complexity.

The Multi-Agent System

6.1 • BaseAgent — the shared contract

Every agent (`agents/base_agent.py`) is an ABC with a `name`, a list of ChromaDB `collections` it's allowed to search, an abstract `run(query, race, year)`, and a protected `_retrieve()` helper that calls the shared retriever, scoped to only that agent's assigned collections. This is a clean instance of the **Template Method** pattern — shared retrieval plumbing in the base class, domain-specific prompting in each subclass.

6.2 • The six agents

TyreAgent

`collections: laps, pitstops`

Tyre compounds, degradation, stint lengths, undercut/overcut windows, pit-stop timing.

WeatherAgent

`collections: weather`

Air/track temperature, humidity, wind, rain risk, and how conditions feed into tyre choice.

RadioAgent

`collections: radio`

Extracts strategic intent from team-radio transcripts — driver complaints, box calls, engine-mode instructions.

RivalsAgent

`collections: laps, pitstops`

Competitor-focused: rival tyre age, degradation trend, predicted pit windows, undercut/overcut threats.

CircuitAgent

`collections: radio, laps`

Track-specific — Safety Cars, Virtual Safety Cars, yellow flags, overtaking difficulty, position-change likelihood.

DataAgent

`collections: (none – queries SQLite directly)`

The text-to-SQL specialist — see Chapter 7 for a full breakdown.

Every semantic agent (all but DataAgent) shares the same **five-rule system-prompt skeleton**: answer only from supplied context; say the fallback phrase if the context is insufficient; be concise and use correct F1 terminology; never expand a driver code into a guessed full name; and — critically — never state an aggregate/total as if it were exhaustive, because retrieval returns a *sample*, not the full dataset.

6.3 • The Orchestrator — routing

Routing is **LLM-based, not keyword/regex-based**. A dedicated system prompt lists each agent with a one-line description and asks the router model to output a comma-separated subset (1–3 agents). Two pragmatic guardrails wrap this LLM call:

- **Fallback to TyreAgent** if the router's output doesn't parse to any known agent name at all (defensive parsing of free-form LLM text output).
- **DataAgent is always appended** regardless of what the router picked. The comment in the code is explicit about why: whether a question is "quantitative enough" for SQL is a judgment call the router gets wrong often (e.g. "which team pitted the most?" got routed to tyre/rivals and never reached DataAgent, even though it's a plain `COUNT / GROUP BY`). DataAgent is cheap (one local SQLite round-trip) and self-limiting (it reports "I don't have enough data" for anything outside its schema), so including it unconditionally costs a little latency but never actively hurts a semantic answer.

6.4 • The Orchestrator — synthesis

After all selected agents return, `_synthesize()` makes **one more LLM call** (the larger `llama-3.3-70b-versatile` model) that receives every agent's raw report and must produce one unified, natural answer. Its system prompt encodes several non-obvious rules:

1. Never mention agent names or expose the internal multi-agent process to the user — the per-agent breakdown is shown separately in the UI, so the prose answer should read as one voice.
2. DataAgent's SQL-derived report is **authoritative** for factual lookups (counts/totals/"who won") — when it fully answers the question, state *only* that, and don't blend in team-specific claims from the semantic agents, because their retrieved sample never contains team affiliation and any team-level claim they make is unverified.
3. A real **driver-code** → **full-name legend** is injected (pulled fresh from the `drivers` SQLite table, scoped to the current race/year) so the synthesis step can expand "VER" → "Max Verstappen (Red Bull Racing)" using *real* grounded data instead of letting the LLM guess from its own training-data priors — which get it wrong whenever a season's grid doesn't match what the model "expects" (rookies, mid-season driver swaps).
4. If every agent reported "I don't have enough data to answer that," the synthesis step must output exactly that same fallback string — the "don't know" signal propagates cleanly through the whole pipeline instead of getting smoothed over into a vague, made-up-sounding answer.

Why does synthesis need the larger model? The code comment is candid about this: reconciling several (sometimes overlapping, sometimes conflicting) specialist reports into one correct answer needs more reasoning than the fast default model reliably gives — it was observed to ignore an unambiguous DataAgent answer in favour of an unrelated driver code mentioned in passing by a different report. That's a genuine empirical finding worth quoting directly if asked about model-selection trade-offs.

Text-to-SQL: the Data Agent

`DataAgent` is architecturally different from every other agent: it has **zero ChromaDB collections** and instead translates the question into a SQLite query against `data/pitwall.db`, executes it, and hands the raw rows back unsynthesized (the orchestrator's own synthesis step turns them into prose later — running a second LLM call here just to re-describe the same numbers would be wasted latency).

Pipeline inside `DataAgent.run()`

- 1 Fetch the live schema (`PRAGMA table_info` per table) and the *actual distinct* `Team` / `Compound` values currently in the DB.
- 2 Prompt `llama-3.3-70b-versatile` to output *only* a raw SQL string, or the literal token `UNANSWERABLE` for non-quantitative questions.
- 3 Guard the output: reject anything whose first keyword isn't `SELECT` / `WITH`, or that contains a `;` (multi-statement) — **before** it ever reaches `sqlite3.execute`.
- 4 Execute; on failure, feed the exact error back to the model and retry once (`MAX_SQL_ATTEMPTS = 2`).
- 5 Format up to 50 result rows as `col=value` pairs (more natural for the downstream synthesis LLM to read than a raw tuple).

Why "give the model the real Team/Compound values" matters

Free-text questions rarely use the exact stored string — a user says "Red Bull" or "Merc," the database stores `"Red Bull Racing"` / `"Mercedes"`. An exact-match `WHERE Team = 'Red Bull'` clause doesn't error, it silently returns **zero rows** — arguably the worst failure mode, because it looks like a real (wrong) answer rather than an obvious error. Handing the model the live enumeration of valid values turns this into a lookup/mapping problem instead of a guess.

Why this is safe against SQL injection

The generated SQL is never string-interpolated with user input into a larger query — the LLM's entire output *is* the query, executed as-is. The safety net is the explicit guard (SELECT/WITH-only, single-statement-only) plus the fact that SQLite has no equivalent of stacked queries via a plain `cursor.execute()` call the way some drivers allow — but the guard is what actually prevents an LLM-hallucinated `DROP TABLE` from ever running, since the model is never told the DB is read-only in a way that's *enforced* rather than just *requested* in the prompt.

Interview gold: This is a strong answer to "how do you make an LLM-generated SQL query safe?" — defense in depth: (1) prompt-level rule "SELECT only," (2) schema/value grounding to reduce wrong-but-valid queries, (3) a hard code-level guard that doesn't trust the prompt rule to have worked, (4) one bounded self-correction retry with the real error message fed back, (5) a row cap so a pathological query can't blow up context or memory.

Backend API — FastAPI Routes & Design Patterns

8.1 • Two categories of route

The API deliberately splits routes into two families with very different reliability/latency characteristics:

LLM-driven (agentic)

POST /ask • GET /brief • GET /debrief

Go through the Orchestrator — open-ended natural-language questions, non-deterministic, several seconds of latency, can say "I don't know."

SQL-direct (deterministic)

GET /race-stats • /race-results • /pit-stops • /weather-summary • /races

Straight `sqlite3` queries, no LLM call at all — exact facts (winner, podium, fastest lap, pit-stop log) that must never be wrong or hallucinated, and load near-instantly.

`/schedule`, `/schedule/next`, and `/standings/*` form a third, even simpler category: they just serve pre-fetched JSON files straight off disk (`data/schedule_full_<year>.json`, `data/standings_<year>.json`) with no database query at all.

8.2 • Full route table

ROUTE	METHOD	BACKED BY	PURPOSE
<code>/</code>	GET	—	Liveness probe (used as Render's health check).
<code>/status</code>	GET	ChromaDB	Chunk counts per collection — vector-store health check.
<code>/f1dash/status</code>	GET	local TCP probe	Checks if a local <code>f1-dash</code> live-telemetry container is running on port 3001.
<code>/ask</code>	POST	Orchestrator	Single free-form question → synthesized answer + per-agent breakdown.
<code>/brief</code>	GET	Orchestrator ×5, cached	5 fixed pre-race strategy questions as cards.
<code>/debrief</code>	GET	Orchestrator ×3, cached	3 fixed post-race analysis questions as cards.
<code>/races</code>	GET	SQLite n schedule JSON	Races that actually have ingested data (frontend's race selector source of truth).
<code>/race-stats</code>	GET	SQLite	Winner, podium, fastest lap, compound breakdown, pit stats, grid — pure SQL.
<code>/race-results</code> , <code>/pit-stops</code> , <code>/weather-summary</code>	GET	SQLite	Full classification / pit log / weather averages.
<code>/schedule</code> , <code>/schedule/next</code>	GET	JSON file	Full season calendar incl. future rounds; soonest upcoming race + countdown.
<code>/standings/drivers</code> , <code>/standings/constructors</code>	GET	JSON file	Championship tables.

8.3 • Notable implementation details

- **Shared parsing helper** — `api/race_utils.py::parse_race_label()` splits the frontend's `"Country Year"` label (e.g. `"Saudi Arabia 2025"`) into `(race, year)` via a single regex, reused by every route that needs to scope a query — one canonical implementation instead of five ad-hoc string splits.
- **Process-lifetime caching for brief/debrief** — since the underlying race data never changes once ingested and the questions are fixed, re-running the full multi-agent pipeline (routing + parallel agents + synthesis, five times) on every page view would be pure wasted latency. A simple `dict[str, BriefResponse]` keyed by race label caches results for the life of the process, with an explicit `force_refresh` query param to bypass it.
- **Single shared Orchestrator instance per route module** — agents are pre-instantiated once at import time (`_orchestrator = Orchestrator()`), not per-request, since agent objects are stateless aside from their name/collections.
- `/races` vs `/schedule` — a deliberate distinction: `/races` only lists weekends with real ingested lap data (so the chat/brief/debrief UI never offers a race it can't actually answer questions about); `/schedule` shows the entire official calendar including future rounds, for the "next race" countdown.

- **Pydantic models as the API contract** — every request/response shape lives in `api/models.py`; FastAPI uses them for both runtime validation and automatic OpenAPI/Swagger docs at `/docs`.

Frontend Architecture

9.1 • Routing & page structure

```
<BrowserRouter>
  <Sidebar /> <NavBar />
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/races" element={<Races />} />
    <Route path="/races/:year/:round" element={<RaceWeekend />} />
    <Route path="/standings" element={<Standings />} />
    <Route path="/chat" element={<Chat />} />
  </Routes>
</BrowserRouter>
```

Because this is client-side routing, a hard refresh on `/races/2025/8` would 404 on a naive static host — Vercel needs an explicit SPA rewrite (`frontend/vercel.json`) that sends every path back to `index.html` so React Router can take over client-side.

9.2 • Global state — `AppContext`

A single React Context (`context/AppContext.tsx`) is the app's only global state, deliberately kept small:

- `races` — fetched once from `GET /races` on mount, mapped into a richer client-side `Race` shape that adds a `circuitKey` (via a static country→circuit lookup table used to render an SVG track outline per circuit).
- `status` — polled from `GET /status` every 60 seconds; drives an "online/offline" indicator so the UI can tell the user the backend is unreachable rather than just spinning forever.
- `latestRace` — derived by sorting `races` by year/round descending; used as the default selection in the Chat page.

Both fetch effects fail *silently* into a safe default (a hardcoded `SEED_RACE` = Monaco 2025) rather than throwing — a deliberate choice so a backend hiccup degrades the UI instead of white-screening it.

9.3 • Typed API client — `lib/api.ts`

Every backend endpoint has a matching typed function (`askQuestion`, `getRaceStats`, `getSchedule`, ...) and a matching TypeScript `interface` mirroring the Pydantic response model on the backend — the closest thing this project has to a shared schema without a code-generation step. A shared `fetchWithTimeout()` wraps every call in a 30-second `AbortController` timeout, and a custom `ApiError` class normalizes both network failures and non-2xx JSON error bodies into one catchable type.

```
const BASE_URL = import.meta.env.VITE_API_BASE_URL || 'http://localhost:8000';
```

This one line is the whole story of frontend/backend decoupling: in dev it defaults to the local uvicorn server; in production, Vercel injects `VITE_API_BASE_URL` pointing at the Render URL at *build* time (Vite inlines

`import.meta.env.*` values into the static bundle — there's no runtime env lookup in the browser).

9.4 • Key pages

- **Chat** — the direct window into the agent system. Renders per-agent result pills (color/icon per agent name, e.g. red for Tyre, blue for Weather) alongside the synthesized answer, so a user can see *which* specialists contributed.
- **RaceWeekend** (`/races/:year/:round`) — tabbed view (results / qualifying / brief / debrief) for one race weekend; falls back to a "not yet ingested, here's a countdown" state (via `getSchedule`) for future rounds that don't have data yet.
- **Standings** — Drivers'/Constructors' Championship tables from the JSON-backed standings routes.
- **Home** — landing dashboard (next race countdown, top drivers, etc.).

LiveMap / f1-dash: One component (`LiveMap.tsx`) embeds an `<iframe>` pointing at a separate, optional local Docker service (`f1-dash` , a live telemetry visualizer) for real-time car positions during a live session. It's not part of this repo, degrades to a clean "offline" state when unreachable (polled via `/f1dash/status`), and its URL is env-configurable (`VITE_F1DASH_URL`) — a good example of integrating an optional external tool without coupling the core app to it.

CHAPTER 10

Database Design

10.1 • SQLite schema (`data/pitwall.db`)

Built by `utils/db_manager.py::init_db()`, which drops and fully rebuilds the file from the season's CSVs every time it's run (idempotent, not incremental) using `DataFrame.to_sql()`. Five tables, every one tagged with `Race` + `Year`:

TABLE	GRAIN	KEY COLUMNS
<code>laps</code>	one row per driver-lap	Driver, Team, LapNumber, LapTime, LapTimeSeconds*, Compound, TyreLife, Position, PitInTime, PitOutTime, Race, Year
<code>pitstops</code>	one row per pit stop	Driver, Team, LapNumber, PitInTime, PitOutTime, Compound, TyreLife, Race, Year
<code>weather</code>	one row per weather sample	AirTemp, Humidity, Pressure, Rainfall, TrackTemp, WindDirection, WindSpeed, Race, Year
<code>drivers</code>	one row per driver per race weekend	Driver (code), FullName, Team, FinishPosition, Race, Year
<code>qualifying</code>	one row per grid slot	GridPosition, Driver, FullName, Team, Q1, Q2, Q3, Race, Year

* `LapTimeSeconds` is a derived column added at ingest time — FastF1 stores lap times as pandas `Timedelta` strings, which SQL can't do arithmetic/aggregation on directly, so it's pre-converted to a plain float for `AVG()` / `ORDER BY` queries.

`drivers` exists purely to solve one recurring problem: **laps and pitstops only ever store 3-letter driver codes** (e.g. `"ANT"`), never full names — so any "who won" / "who is X" question needs a `JOIN ... ON Driver=Code AND Race=... AND Year=...` against this table to answer with a real, grounded name instead of the LLM guessing.

10.2 • ChromaDB collections

COLLECTION	CHUNK SOURCE	USED BY
<code>laps</code>	per driver-lap sentence	Tyre, Rivals, Circuit
<code>weather</code>	windowed 10-row averages	Weather
<code>radio</code>	per-utterance transcript	Radio, Circuit
<code>pitstops</code>	per pit-stop sentence	Tyre, Rivals

All four share one embedding function instance and one `PersistentClient`, stored on disk at `vectorstore/` — roughly **107 MB** total (HNSW index files + a SQLite-backed metadata store that ChromaDB manages internally).

A real gotcha worth mentioning if asked about debugging: There are *two* differently-located Chroma stores in this repo's history — `data/chroma_db/` (an old, now-unused location, still present but dead) and `vectorstore/` at the repo root (what `utils/vectorstore.py` actually opens). `reingest.py` has an explicit code comment warning about this: deleting collections at the wrong path silently no-ops, leaving stale data in place with no error. This is a good real example of "quiet failure modes" in systems with more than one place data *could* live.

Prompt Engineering & Anti-Hallucination Design

This is arguably the most interview-relevant chapter — every rule below exists because a *specific* failure mode was observed and then closed off at the prompt or code level.

11.1 • The "don't know" escape hatch

Every agent's system prompt mandates the exact fallback string `"I don't have enough data to answer that."` when context is insufficient, and the orchestrator's synthesis prompt propagates that same string upward if every agent hit it. Standardizing on one literal string (rather than each agent phrasing "I don't know" differently) means the frontend and any downstream logic can treat "no answer" as a single, detectable case instead of trying to pattern-match varied hedging language.

11.2 • Driver-code grounding, not name-guessing

Every prompt explicitly forbids expanding a 3-letter driver code (e.g. `VER`, `ANT`) into a full name from the model's own knowledge — "code-to-driver assignments for this season may not match what you'd expect, and guessing produces wrong names." The fix is architectural, not just a prompt instruction: the orchestrator queries the real `drivers` table and injects a "use these EXACT names, do not guess" legend into the synthesis prompt only — agents stay code-only, and name resolution happens exactly once, from ground truth.

11.3 • "Your context is a sample, not the full dataset"

Every semantic agent's prompt contains a rule that is unusually specific for a system prompt: never state a total/tally/aggregate as if exhaustive, and never attribute a stat to a team unless the retrieved chunk text itself names that team (because team affiliation is never present in lap/pitstop chunk text at all). This single rule is what makes the DataAgent/semantic-agent split in Chapter 7 necessary rather than optional — it's an explicit, written acknowledgement that vector search top-k results are a *sample*, and treating a sample as ground truth for a COUNT-style question is a specific, named failure mode this project engineered around rather than ignored.

11.4 • Synthesis defers to SQL for factual questions

Rule 5 of the synthesis prompt (Chapter 6.4) is the sharpest example of designing *trust hierarchy* into a prompt: when DataAgent's SQL-derived report fully answers a factual question, state *only* that — don't let a semantic agent's unverifiable "supporting" detail get blended in, even though blending sounds more thorough. More detail is not always more correct.

11.5 • Model selection as a prompt-engineering lever

Not every LLM call uses the same model — this is itself a form of prompt/pipeline engineering:

CALL SITE	MODEL	REASONING
Router (<code>Orchestrator.route</code>)	<code>llama-3.1-8b-instant</code>	Simple classification task, needs speed over depth.
Each specialist agent's answer	<code>llama-3.1-8b-instant</code>	Narrow, well-scoped context per agent; speed matters since up to 4 run in parallel.
Text-to-SQL generation	<code>llama-3.3-70b-versatile</code>	More prone to hallucinating a wrong column/table than narrative generation — correctness > speed.
Final synthesis	<code>llama-3.3-70b-versatile</code>	Reconciling multiple, sometimes-conflicting reports needs more reasoning than the fast model reliably gave (observed failure: ignoring an unambiguous SQL answer).

11.6 • Groq client resilience

`utils/groq_client.py` instantiates the Groq SDK client with `max_retries=1`, down from the SDK's own default of 2. The reasoning in the code comment: the SDK's default retry logic respects Groq's `Retry-After` header on HTTP 429s, which on the free tier can be 10+ seconds — with 2 retries that's 20-30+ seconds of pure dead wait on a *single* LLM call before it even fails. One retry is enough to smooth over a transient blip; the orchestrator already tolerates one agent failing outright (Chapter 3), so a sustained rate limit degrades gracefully rather than stalling the whole request.

Deployment & DevOps

12.1 • Split-repo deployment

Backend → Render (Web Service via `render.yaml` blueprint), frontend → Vercel (Vite static build, root directory `frontend/`). The two are decoupled entirely through one env var — `VITE_API_BASE_URL` on the Vercel side pointing at the Render URL, and `CORS_ORIGINS` on the Render side allow-listing the Vercel URL.

12.2 • Committing the vector store — a deliberate trade-off

The ChromaDB store (`vectorstore/`, ~107 MB) was initially `.gitignore`d since it's *built* data. But Render's deploy is a plain `git clone` + build — it never runs the ingestion pipeline, and rebuilding embeddings at deploy time would mean re-hitting the FastF1 API for ~29 race weekends on every deploy (slow, fragile, dependent on an external service staying up). The chosen fix: un-ignore and commit `vectorstore/` directly. No single file exceeds GitHub's 100 MB hard limit (two files landed just over the 50 MB *soft* warning threshold, which only prints a warning, not an error). The alternative — a Render persistent disk seeded manually after first deploy — was considered and explicitly rejected for a free-tier/no-cost portfolio deployment, since paid disks aren't available on the free plan and manual seeding adds an error-prone step outside of version control.

12.3 • The production incident: Render OOM on a 512 MB free instance

The single most interview-worthy story in this project. After the first deploy, the Render service crashed with an out-of-memory error on its 512 MB free-tier instance. Root cause diagnosis:

- 1 Locally (Windows), `pip install sentence-transformers` pulls torch's **CPU-only** wheel automatically — PyPI only hosts CPU builds for Windows at all.
- 2 On Render (Linux), the exact same `pip install` command resolves torch's **default CUDA-enabled** build from PyPI — dragging in `nvidia-cublas`, `nvidia-cudnn`, `nvidia-cusolver`, and several other multi-hundred-MB packages nobody asked for, since there's no GPU on Render at all.
- 3 That alone balloons the install to multiple GB and blows past 512 MB before a single request is even served.

The fix — force-install the CPU-only wheel from PyTorch's own package index *before* the rest of the dependencies, so the later `sentence-transformers` install sees torch's version requirement already satisfied and skips pulling the CUDA build entirely:

```
# render.yaml
buildCommand: >
  pip install torch --index-url https://download.pytorch.org/whl/cpu
  && pip install -r requirements-api.txt
```

Why this is a strong interview story: It's a real, non-obvious, platform-specific bug (works locally, breaks in prod) with a root cause that required knowing *why* pip resolves differently per-OS, and a fix that's minimal and targeted rather than "just upgrade the instance" (which costs money and doesn't fix the actual waste). It demonstrates reading dependency resolution behavior, not just reading a stack trace.

12.4 • Trimming the deploy dependency surface

Grepping every import across `api/`, `agents/`, and `utils/` confirmed the live API **never imports** `fastf1`, `faster-whisper`, `yt-dlp`, or `anthropic` — those are exclusively used by the offline `ingest/` scripts. A separate `requirements-api.txt` (fastapi, uvicorn, chromadb, sentence-transformers, groq, python-dotenv, pandas) is what Render actually installs, cutting build time and image size versus the full `requirements.txt` used for local ingestion work. This also fixed a latent bug: `pandas` was imported directly in several files but was only ever present *transitively* (via fastf1) in the original requirements file — trimming fastf1 out would have silently broken the slim install had pandas not been pinned explicitly.

12.5 • Other deploy-friendliness fixes made

- **CORS** — moved from a hardcoded `allow_origins=["*"]` to a `CORS_ORIGINS` env var (comma-separated allowlist, defaults to `*` so local dev is unaffected).
- **Frontend API base URL** — was hardcoded to `http://localhost:8000`; switched to `import.meta.env.VITE_API_BASE_URL` with that same value as the dev fallback.
- **SPA rewrite** — `frontend/vercel.json` routes every path to `index.html` so client-side routes survive a hard refresh.
- **Host/port binding** — `uvicorn api.main:app --host 0.0.0.0 --port $PORT` (Render injects `$PORT` at runtime; binding to `0.0.0.0` rather than `127.0.0.1` is required for the platform's reverse proxy to reach the container).

Key Engineering Decisions — a Curated "Why" List

These are the decisions most likely to come up as "walk me through a trade-off you made" — each phrased as decision → alternative considered → why this one won.

Q Why six narrow agents instead of one big RAG prompt?

A single system prompt covering tyres + weather + radio + rivals + circuit simultaneously has to split its attention across unrelated context in one call, which measurably produces shallower, less domain-precise answers than a specialist with a narrow collection scope and a system prompt written entirely around one concern. The cost is more LLM calls per question; the mitigation is running them concurrently (Chapter 3) and routing so only relevant agents fire.

Q Why LLM-based routing instead of keyword matching?

Keyword rules are brittle against paraphrase ("box this lap" vs. "pit stop timing" vs. "when should he come in") and don't generalize to new phrasings without manual rule maintenance. An LLM router trades a small amount of latency/cost for much better recall on intent, at the cost of occasional misclassification — which is why DataAgent is force-included regardless of the router's decision (Chapter 6.3), a pragmatic patch over a probabilistic component's known weak spot.

Q Why SQLite *and* ChromaDB, not just one store?

They answer fundamentally different question shapes. SQLite/SQL gives exact, provably-complete answers to aggregate questions ("how many," "who won," "average lap time"). ChromaDB gives semantic recall over unstructured/narrative text (radio commentary, per-lap descriptions) where there's no fixed schema to filter on. Using vector search alone for aggregate questions is unreliable because top-k retrieval is a sample, not an exhaustive scan — a fact baked directly into every agent's system prompt.

Q Why threads (not asyncio or multiprocessing) for parallel agents?

Each agent call is I/O-bound (waiting on an HTTP round-trip to Groq/ChromaDB), not CPU-bound, so Python's GIL doesn't meaningfully limit concurrency here — `ThreadPoolExecutor` gets the parallelism with far less code than converting the whole call chain to `async`/`await`, and without multiprocessing's IPC/serialization overhead for what's fundamentally a "wait on a few network calls" problem.

Q Why commit built data (SQLite DB + vector store) to git instead of building it at deploy time?

The ingestion pipeline depends on an external, sometimes-flaky data source (FastF1's upstream API) and takes real time to run across ~29 race weekends. Coupling every deploy to that pipeline succeeding would make deploys slow and occasionally fail for reasons unrelated to the actual code change being shipped. Since the underlying F1 data for a past race weekend never changes, building once locally and committing the artifact is simpler and more reliable than rebuilding it on every deploy — the standard "build once, deploy the artifact many times" principle, just applied to a data artifact instead of a compiled binary.

Q Why does the API never call an LLM for /race-stats but does for /brief?

Anything with one exact, computable answer (winner, fastest lap, pit-stop count) is answered with SQL and never touches an LLM — faster, free, and literally cannot hallucinate. Anything requiring synthesis/interpretation across multiple data types (tyre strategy reasoning, "why" questions) genuinely needs the LLM+RAG pipeline. Routing each kind of question to the cheapest mechanism that can answer it correctly is a recurring theme in this codebase.

CHAPTER 14

Glossary

RAG (Retrieval-Augmented Generation)

Feeding an LLM relevant retrieved documents as context alongside a question, so it answers from supplied facts instead of (or in addition to) its own training data — reduces hallucination and allows answering about data the model was never trained on.

Embedding

A fixed-length vector of floats (here, 384 numbers) representing a piece of text's meaning, produced by an embedding model, such that semantically similar text produces vectors that are close together in that vector space.

Cosine distance / similarity

A measure of the angle between two vectors, used to rank how semantically similar two pieces of text are. Distance = $1 - \text{similarity}$ for normalized vectors; PitWall's retriever filters out any match with cosine distance above a threshold (i.e., below a similarity floor).

Chunking

Splitting source data into small, independently-retrievable units before embedding — the granularity choice (one chunk per lap vs. per race, for example) directly affects retrieval precision.

Vector database

A database optimized for storing embeddings and performing fast nearest-neighbour search over them (here, ChromaDB, using an HNSW index under the hood).

HNSW

Hierarchical Navigable Small World — the approximate-nearest-neighbour graph algorithm ChromaDB uses internally to make vector search sub-linear instead of comparing against every stored vector.

Text-to-SQL

Using an LLM to translate a natural-language question directly into an executable SQL query against a known schema, rather than retrieving semantically similar text.

Multi-agent system

An architecture where multiple LLM-backed "agents," each with a narrower role/scope/prompt, collaborate (in parallel and/or in sequence) to answer a broader question, typically coordinated by an orchestrator that routes work and reconciles their outputs.

Orchestration

The coordination layer that decides which agents to invoke, runs them, and combines their outputs into one final response.

System prompt / user prompt

In a chat-style LLM API, the system prompt sets persistent role/rules/behaviour for the whole conversation; the user prompt is the specific input (here, retrieved context + the question) for this turn.

Hallucination

An LLM stating something confidently that isn't actually supported by its input/training — the central failure mode this project's prompt rules (Chapter 11) are engineered to prevent.

CORS (Cross-Origin Resource Sharing)

A browser security mechanism that blocks a webpage from calling an API on a different origin (domain/port) unless that API explicitly allows it via response headers — why the FastAPI backend needs a `CORSMiddleware` allowlist to be callable from the Vercel-hosted frontend.

SPA (Single-Page Application)

A frontend architecture where one HTML page loads once and JavaScript handles all subsequent "navigation" client-side (via a router) without full page reloads — requires server-side rewrite rules so deep-link URLs still resolve on a hard refresh.

ASGI

Asynchronous Server Gateway Interface — the async successor to WSGI; the protocol FastAPI and uvicorn speak to each other, enabling async request handling.

Blueprint (Render)

Render's mechanism for defining infrastructure-as-code in a `render.yaml` file committed to the repo, so a service's build/start commands and env vars are version-controlled rather than manually configured through a dashboard.

Interview Q&A Bank

System design

Q How would this scale to 100x traffic?

The bottleneck today is Groq API latency and rate limits, not compute — each `/ask` triggers 3–6 sequential/parallel LLM calls. At scale: cache brief/debrief answers (already done, but could move to Redis/shared cache across instances instead of in-process dict, which doesn't survive a restart or scale past one instance); add a request queue/backpressure for `/ask`; consider a streaming response so the user sees partial agent results as they land instead of waiting for full synthesis; and horizontally scale the stateless FastAPI service behind a load balancer since it holds no per-request server state.

Q What's a single point of failure here?

Groq — every LLM call in the system goes through one provider. A production hardening step would add a fallback provider or at least a circuit breaker so a Groq outage degrades to the deterministic SQL-only routes (`/race-stats` etc.) rather than the whole agent-based experience failing.

RAG & retrieval

Q How do you evaluate retrieval quality?

Honestly, this project doesn't have a formal offline eval set (e.g. labeled query→relevant-chunk pairs with recall@k metrics) — retrieval tuning here was empirical (distance-threshold and top_k values were adjusted based on observed answer quality, per the code comments). A natural next step for a production version: build a small golden-question set per agent and measure whether the expected chunk appears in the top-k results before and after any retrieval change.

Q Why chunk per-lap instead of per-race or per-driver?

Granularity is a precision/recall trade-off. Per-race chunks would be too coarse — a single embedding can't represent 70+ laps of varied context well, and retrieval couldn't distinguish "what happened on lap 32" from "what happened on lap 5." Per-lap chunks let the vector search pinpoint the exact moment being asked about, at the cost of more total chunks to store/search (mitigated by metadata filtering on race/year before ranking).

Multi-agent / orchestration

Q What happens if two agents disagree?

The synthesis LLM call is explicitly instructed to reconcile contradictions and treat DataAgent's SQL-derived answer as authoritative for factual questions, silently dropping unverifiable specifics from the semantic agents rather than trying to merge them. For genuinely open-ended/opinion-style questions where there's no ground truth to defer to, the synthesis prompt just asks for one consolidated, non-contradictory answer — there's no explicit voting or confidence scoring mechanism today, which is a fair limitation to name if asked (see Chapter 16).

Q Why not use a framework like LangChain/LlamaIndex/CrewAI?

The retrieval pipeline, agent structure, and orchestration logic here are all simple enough (a handful of well-understood pieces: embed → filter → rank → prompt → call) to hand-roll in well under a thousand lines of plain Python, with full visibility into exactly what each call does and full control over prompt content — no framework abstraction to fight when a specific behaviour (like the "DataAgent always included" patch) needs to be layered on top of a general routing mechanism.

Backend / API

Q Is this API stateless? What state does exist?

Yes, per-request — no session/auth/user state at all. The only server-side mutable state is the in-process brief/debrief cache (a plain dict, reset on restart, not shared across instances) — everything else (SQLite, ChromaDB) is read-only at request time, written only by the offline ingestion pipeline.

Q How is user input validated?

Two layers: FastAPI/Pydantic validates request shape (types, required fields) before a handler even runs; and for the one place free-form text becomes something executable — DataAgent's generated SQL — there's an explicit code-level guard rejecting anything that isn't a single read-only SELECT/WITH statement (Chapter 7), independent of whatever the prompt asked the model to do.

Frontend

Q How does the frontend know the backend is down?

`AppContext` polls `GET /status` every 60 seconds; a failed fetch flips a boolean `online` flag in context, which components can use to show an offline indicator, while everything else in the UI keeps working off cached/seed data rather than breaking outright.

Q Why TypeScript interfaces mirroring the Pydantic models by hand instead of code-generation (e.g. `openapi-typescript`)?

A fair critique, and a real next-step improvement: FastAPI already emits a full OpenAPI schema at `/openapi.json` for free, so generating the TS types from it instead of hand-maintaining `lib/api.ts` would remove an entire class of "backend changed a field, frontend didn't" bugs. It wasn't wired up yet in this project — worth naming honestly if asked what you'd change (see Chapter 16).

"What Would You Improve?" — Honest Limitations

Every senior interviewer eventually asks this. Naming real limitations you already understand reads as far stronger than pretending the project has none.

NO EVAL HARNESS**NO AUTH****SINGLE LLM VENDOR****IN-MEMORY CACHE ONLY**

- **No formal retrieval/answer evaluation set.** Threshold and top_k values were tuned empirically; a golden dataset of question→expected-chunk/answer pairs with automated scoring would catch regressions when prompts or chunking change.
- **No authentication/rate limiting on the public API.** Anyone with the Render URL can call `/ask` and consume Groq quota — fine for a portfolio deployment, not for a real product without at minimum per-IP rate limiting.
- **Single LLM provider (Groq) with no fallback** — an outage there takes down every agentic route at once (SQL-only routes would still work).
- **In-process, non-persistent caching** — the brief/debrief cache resets on every Render restart/redeploy and isn't shared if the service ever scaled to multiple instances; a shared cache (Redis) would fix both.
- **No streaming responses** — the user waits for the full multi-agent + synthesis pipeline (several seconds) before seeing anything; streaming partial agent results as they land would improve perceived latency significantly.
- **Hand-maintained frontend/backend type contract** — generating TypeScript types from FastAPI's own OpenAPI schema would remove a class of silent drift bugs.
- **No confidence/voting mechanism between agents** — synthesis reconciles reports via prompt instructions, not a structured scoring or agreement mechanism; fine at this scale, a real limitation at higher stakes.
- **Ingestion is a fully offline, manual step** — there's no automated pipeline that detects and ingests a newly-completed race weekend; it's a script run by hand.

Closing framing for an interview: Ending on "here's what I'd build next and why, in priority order" turns a limitations question into a demonstration of judgment about what actually matters for a system at this scale versus what would be premature optimization for a portfolio project — which is itself a valuable signal to an interviewer.